



Bidaw: Enhancing Key-Value Caching for Interactive LLM Serving

via Bidirectional Computation–Storage Awareness



Shipeng Hu · Guangyan Zhang · Yuqi Zhou · Yaya Wei · Ziyang Zhong · Jike Chen
Tsinghua University · CUGB · China Telecom

FAST '26 Paper Seminar

발표자 : _____ · 일시 : _____

발표 순서

- 1 **Background** Interactive LLM serving 과 KV cache 역할
- 2 **Motivation** KV loading 병목과 워크로드 관찰 세 가지
- 3 **Key Idea** Bidirectional computation-storage awareness
- 4 **Mechanism 1** I/O-aware request scheduling (compute 측)
- 5 **Mechanism 2** Previous-answer-based eviction (storage 측)
- 6 **Mechanism 3** Storage-efficient tensor caching
- 7 **Evaluation** 모델 5 종에 걸친 핵심 plot 7 개
- 8 **Discussion** 한계와 일반화 가능성

Interactive LLM serving — 멀티 라운드 대화 패턴

사례

- Replika · Duolingo · 챗봇 — 사용자와 LLM 이 번갈아 주고받는 대화
 - 사용자 한 명당 평균 22.4 라운드 (P90 = 45 라운드)
 - 한 대화 지속 시간 평균 수 분, 길면 60 분 이상

기술적 핵심

- N 번째 라운드 답변 = 0~N-1 라운드의 KV 를 재사용
- 재계산은 GPU FLOPs 낭비 → caching 이 필수

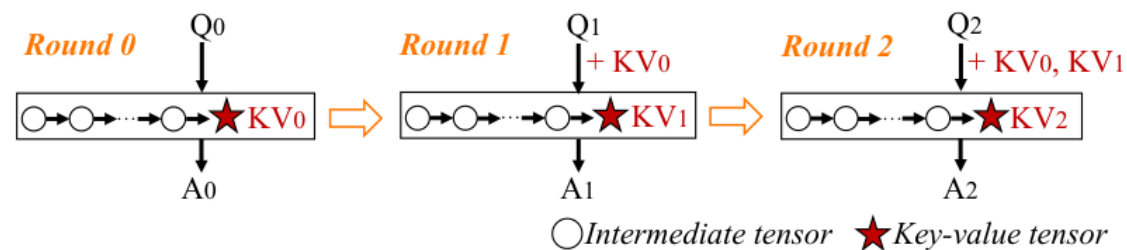


Figure 1. 각 라운드의 답변은 이전 라운드의 KV tensor 를 참조한다

KV cache 역할 — attention 재계산을 피하는 핵심 자료구조

왜 KV cache 가 필요한가

- 재계산을 피하면 응답 latency 가 그만큼 짧아진다
- 저장 비용이 큼 → 효율적 관리가 시스템 설계의 첫 과제
- vLLM, PagedAttention 같은 가속 시스템도 결국 KV cache 관리가 중심

이 논문이 다루는 KV 는 ?

- ▶ 한 사용자의 여러 라운드 대화 KV 를 보존 — 사용자 간 KV 공유는 범위 밖

Self-attention 요점

$$\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d}) \cdot V$$

- ▶ 새 토큰 1 개의 Q 만 새로 계산
- ▶ 과거 토큰의 K, V 는 그대로 → 재사용 가능

토큰 t 개 · layer L · head h · head_dim d 일 때

$$\text{KV size} = 2 \cdot t \cdot L \cdot h \cdot d \cdot \text{sizeof}(\text{dtype})$$

예: OPT-13B 에서 2,048 토큰이면 약 0.78 GB / user

왜 GPU 메모리에 다 담을 수 없는가

- **80GB A800 GPU 1 장** → KV 가 GPU 메모리를 곧바로 채움
 - 도착률 **30 users/min** 에서 OPT-13B 의 동시 캐싱 KV ≈ 480 GB
 - LLM 서버의 **host memory** $\approx$ GPU 메모리의 1.6-3.2 배
 - → **host memory** 도 부족 → capacity SSD 까지 같이 써야 함

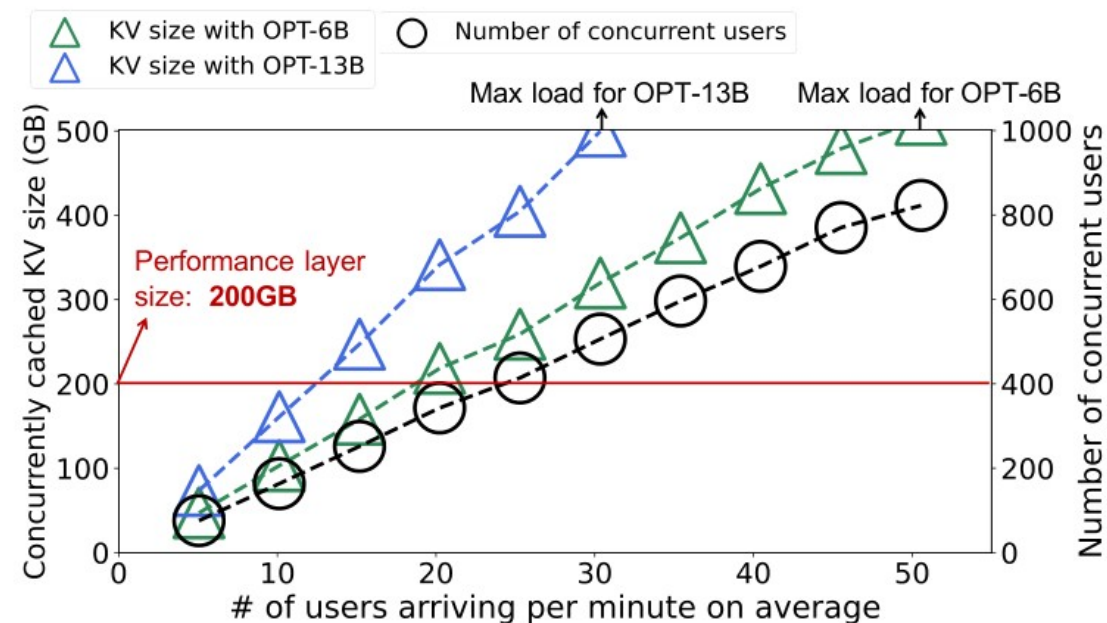


Figure 5. 사용자 도착률 vs 동시 캐싱 KV 총량 (perf layer 200GB 기준)

동시 캐싱 KV 가 perf layer 를 **최대 3.91 배** 초과

→ 2-tier (host memory + SSD) 캐싱이 불가피

기존 솔루션 — Two-tier storage caching

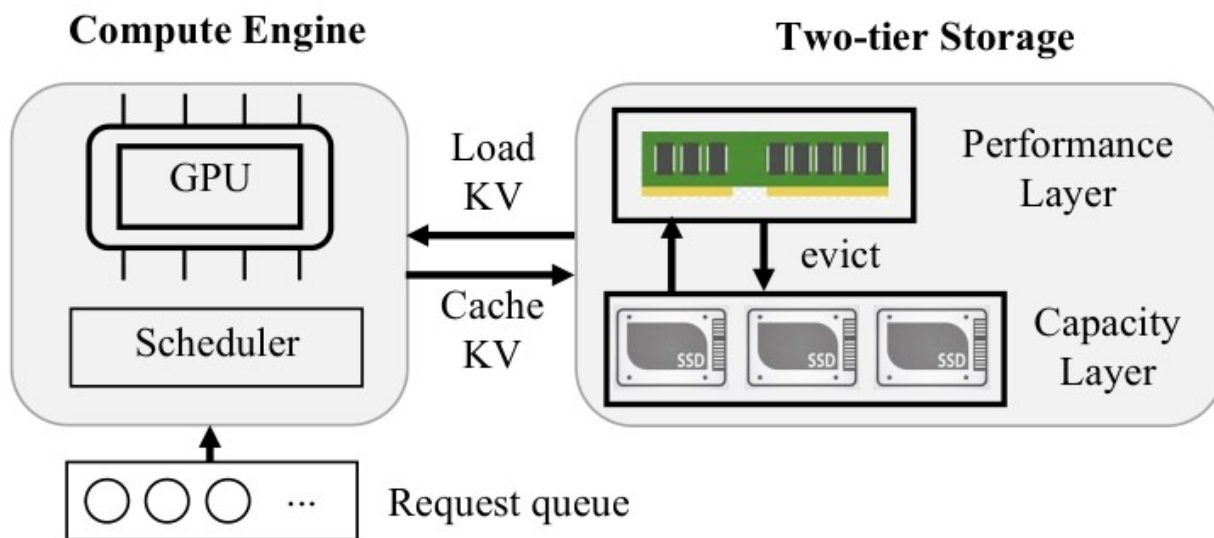


Figure 2. Compute engine 과 two-tier storage 의 구조

Performance layer

Host DRAM — 빠르지만 용량이 작음

Capacity layer

SSD (RAID-5, 이 논문은 1.5 GB/s)

주요 선행 시스템

- ▶ **CachedAttention** [ATC '24] queue-enhanced LRU eviction
- ▶ **FlashGen** [FAST '25] inclusive caching + GPU-fit scheduling
- ▶ **HCache** [EuroSys '25] intermediate activation 캐싱

쓰기는 백그라운드로 빠졌지만, 읽기 (load) 는 여전히 **critical path** 위에 남아 있다

KV loading 자체가 병목 — 이상치 대비 큰 격차

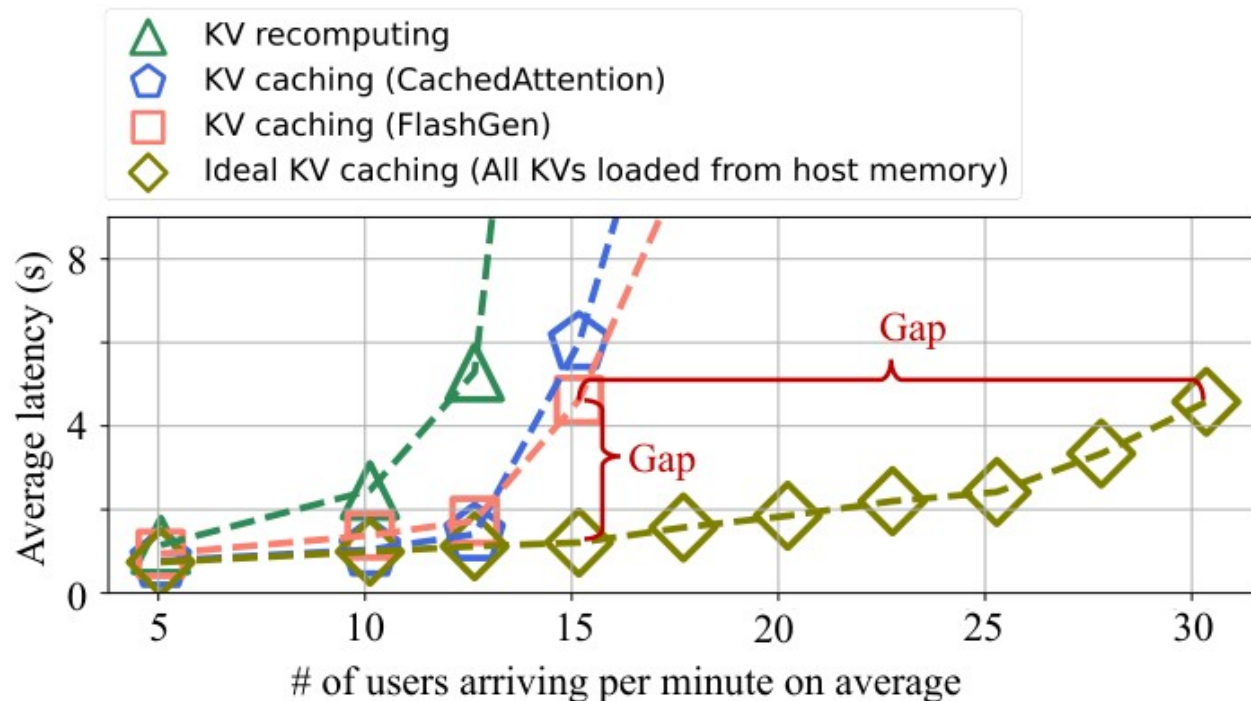


Figure 3. KV 재계산 / 기존 cache / Ideal cache 의 latency 비교

측정 환경

OPT-13B · A800 80GB · 200GB host · 1.5 GB/s SSD

성능 격차

- ▶ 응답 latency **최대 3.8 배 증가**
- ▶ throughput **최대 2.0 배 감소**

Ideal vs 현실

Ideal = 모든 KV 가 perf layer 에 있다고 가정한 동일 시스템

이 격차의 원인을 풀어내는 것이 이 논문의 출발점이다.

워크로드 — 100 만 라운드 규모의 실서비스 대화 trace

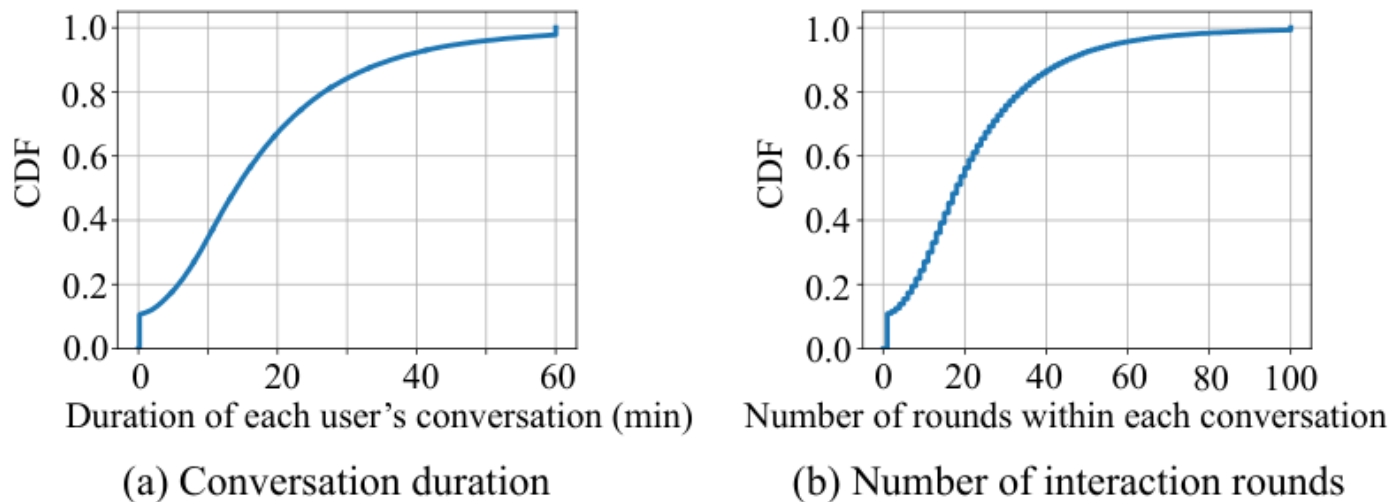


Figure 4. (a) 사용자별 대화 지속시간, (b) 라운드 수 CDF

Trace 출처

China Telecom · 100 만 + 라운드 · 사용자 timestamp 포함

ShareGPT · Mooncake와의 차이

- ▶ ShareGPT: 5.7 라운드 — 인터랙티브 분석엔 짧음
- ▶ Mooncake: 12k+ 토큰 — 길지만 대화 패턴 없음
- ▶ 이 trace: 22.4 라운드 / 36 토큰 — 인터랙티브 워크로드에 잘 맞춤

이어지는 세 슬라이드에서 다룰 내용 → **Observation 1 · 2 · 3**

Observation 1 — 동시 캐싱 KV 가 perf layer 를 크게 초과

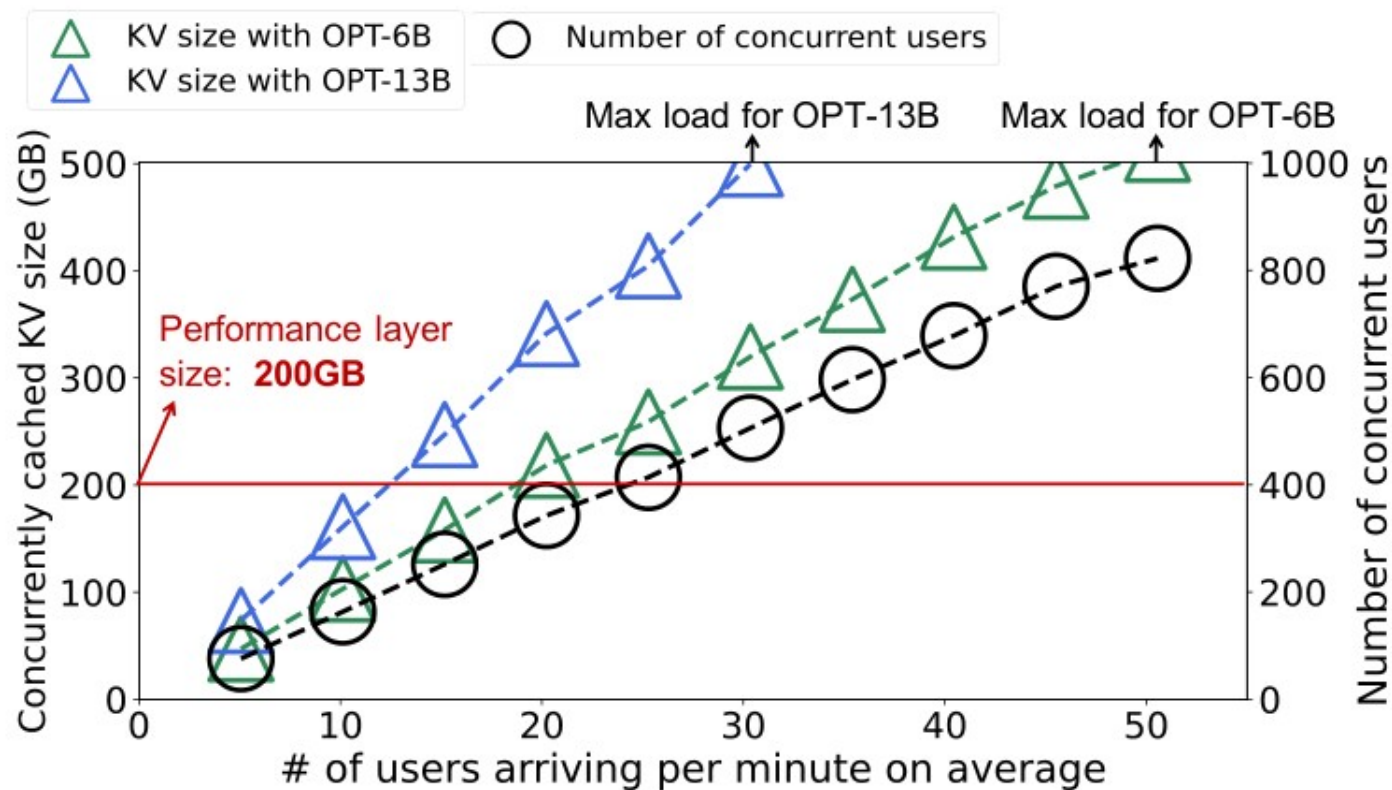


Figure 5. 사용자 도착률 vs 동시 캐싱 KV 총량 / 동시 사용자 수

관찰

- ▶ 한 사용자의 KV 가 22.4 라운드 동안 살아 있음
- ▶ 도착률 ↑ → 동시 사용자 수 선형 증가
- ▶ 30 users/min 에서 480 GB > perf 200 GB

함의

- ▶ perf layer 를 키워도 결국 한계가 있음
- ▶ eviction 이 비효율적이면 capacity layer 로 자주 떨어짐

Observation 2 — KV access 의 temporal locality 가 매우 약함

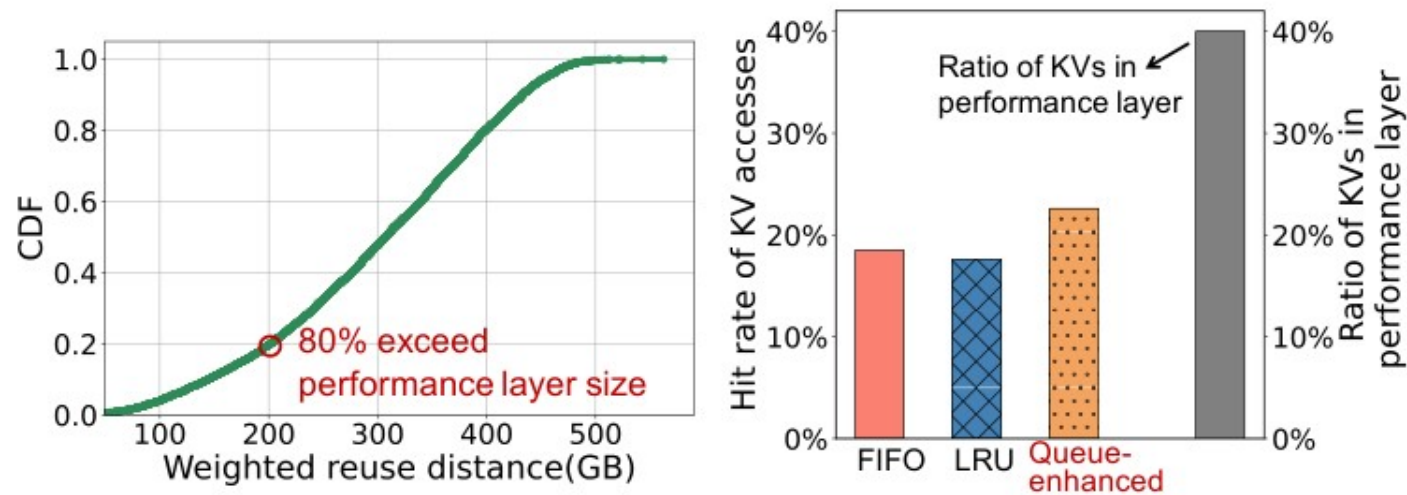


Figure 6. (a) Weighted reuse distance CDF, (b) hit rate

Weighted reuse distance

두 access 사이에 접근된 다른 KV 들의 크기 합

측정치

- ▶ access 의 80% 가 perf 200 GB 를 초과
- ▶ 정책에 상관없이 hit $\approx$ 20%
- ▶ perf 가 40% 를 담아도 hit 은 절반 수준
→ 사용자가 답을 읽는 사이 다른 KV 가 끼어든다

Observation 3 — KV loading 시간 분산이 매우 큼

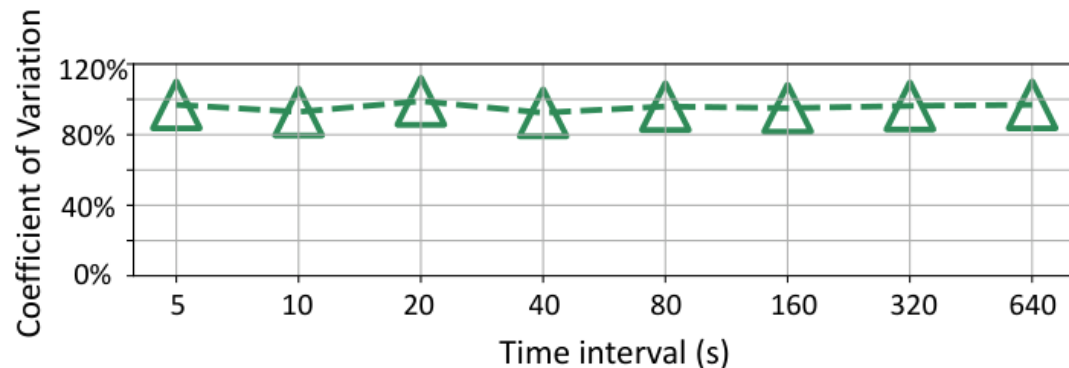


Figure 7. 시간 구간별 KV loading time 의 변동계수 CV

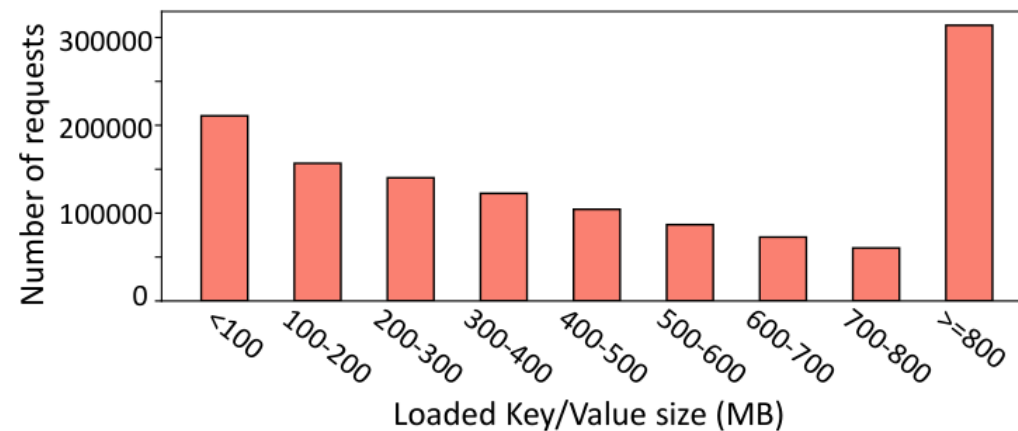


Figure 8. 로드된 KV 크기 분포 (히스토그램)

왜 분산이 크다

▶ host DRAM 과 SSD 의 **bandwidth** 격차 · 요청마다 **KV size** 가 수십 MB ~ 수백 MB 로 다양 · **5 초** 윈도우에서도 **CV > 90%**

함의 — 큰 KV 한 개가 뒤따르는 작은 KV 들까지 막아 GPU 를 놀게 만든다

Root cause — compute engine 과 storage 가 서로를 모름

문제 1. Compute engine 이 storage I/O latency 를 모른다

- ▶ 어느 요청의 KV 가 perf layer 에 있고 , 어느 것이 capacity layer 에 있는지
 - ▶ 어느 요청의 KV size 가 크고 어느 것이 작은지
- FCFS 로 보내면 큰 I/O 하나가 큐 앞을 막아 GPU 가 논다

문제 2. Storage 가 사용자 대화 패턴을 모른다

- ▶ eviction 이 storage 쪽 KV access history 만 사용
 - ▶ 사용자의 다음 질문 시점은 compute 가 만든 답변 길이가 결정
- 정보 부족으로 hit rate 가 20% 수준에 머무름

두 문제 모두 정보 교환이 한 방향에 그치거나 아예 끊겨 있다는 점에서 같다

Bidaw 의 가설 : 양방향 정보 교환을 열어주면 두 문제가 함께 풀린다

핵심 아이디어 — Bidirectional awareness

Compute → Storage

이전 라운드의 **답변 길이**를 storage 에 전달

- ▶ 이 길이가 다음 질문 도착 시점을 가늠하는 신호
- ▶ storage 는 이 신호로 다음 access 의 reuse distance 를 추정
- ▶ hit potential 이 가장 낮은 KV 를 evict

Storage → Compute

각 요청 KV 의 **위치 (layer) 와 크기**를 compute 에 전달

- ▶ compute 가 KV 의 위치와 크기를 알고 dispatch
- ▶ ready / preparing 두 큐로 I/O 길이를 분리
- ▶ GPU 유휴 시간 제거 + 큐잉 지연 감소

시스템 구성 — 3 개 컴포넌트와 양방향 신호

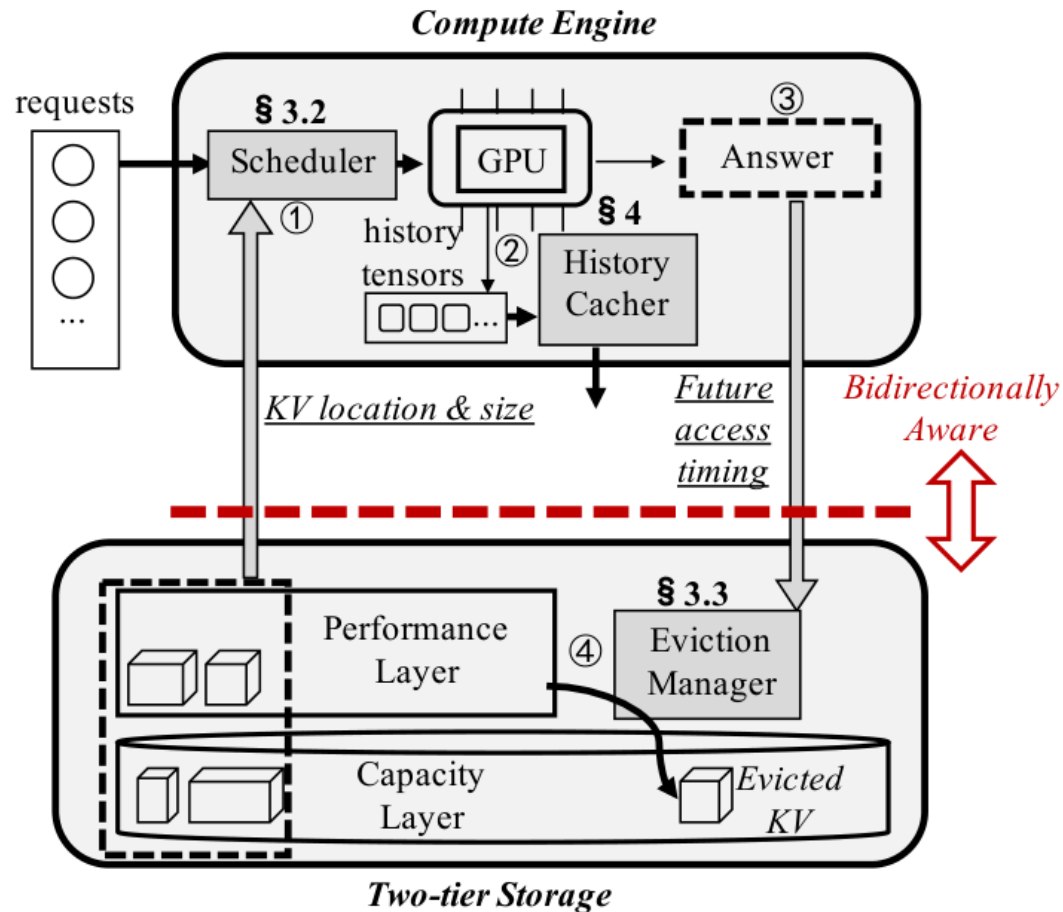


Figure 9. Bidaw 시스템 구조

① Scheduler (§3.2)

storage 에서 KV 위치와 크기를 받아 ready/preparing 큐로 분리

② History Cacher (§4)

GPU 에서 만들어진 history tensor 중 storage-efficient tensor 만 캐싱

③ Eviction Manager (§3.3)

compute 가 보낸 답변 길이로 reuse distance 를 추정하고 hit potential 이 가장 낮은 KV 를 evict

Bidirectional 신호

- ▶ Storage → Compute: KV 의 위치와 크기
- ▶ Compute → Storage: 다음 access 시점 (= 답변 길이)

End-to-end workflow

- ① 사용자 요청이 도착하면 Scheduler 가 KV 의 위치와 크기를 storage 에서 조회
→ Ready queue 또는 Preparing queue 로 보냄
- ② GPU 가 inference 중에 만든 storage-efficient tensor 를 History Cacher 가 보관
→ KV tensor 대신 더 작은 intermediate tensor 를 저장 (MHA 에 한함)
- ③ GPU 응답이 끝나면 답변 토큰 길이를 Eviction Manager 에 전달
→ 다음 KV access 의 reuse distance 분포를 추정
- ④ perf layer 여유 공간이 임계치 아래로 떨어지면 eviction 트리거
→ hit potential 이 가장 낮은 KV 를 capacity layer 로 이동 (inclusive caching)

Mechanism 1 의 동기 — I/O blocking 과 overlap 의 한계



Figure 10. I/O-aware request scheduling 전략 개요

왜 단순한 overlap 으로는 부족한가

- ▶ GPU compute 한 iteration = 수십 ms
- ▶ capacity layer I/O = 수백 ms
- ▶ 다음 layer KV load 와 겹쳐도 시간 차가 너무 큼
- ▶ naive overlap 만으로는 GPU 가 다음 토큰을 기다리며 다시 논다

설계 목표 — 느린 I/O 가 빠른 I/O 를 막지 않으면서, **큰 KV 요청도 starvation 없이 처리한다**

I/O-aware scheduling — Dual queue + disk-HRRN

Dual queue

Ready queue

KV 가 perf layer 에 있는 요청 — FCFS 로 GPU 에 바로 보냄

Preparing queue

KV 가 capacity layer 에 있는 요청 — 백그라운드로 perf layer 에 올림

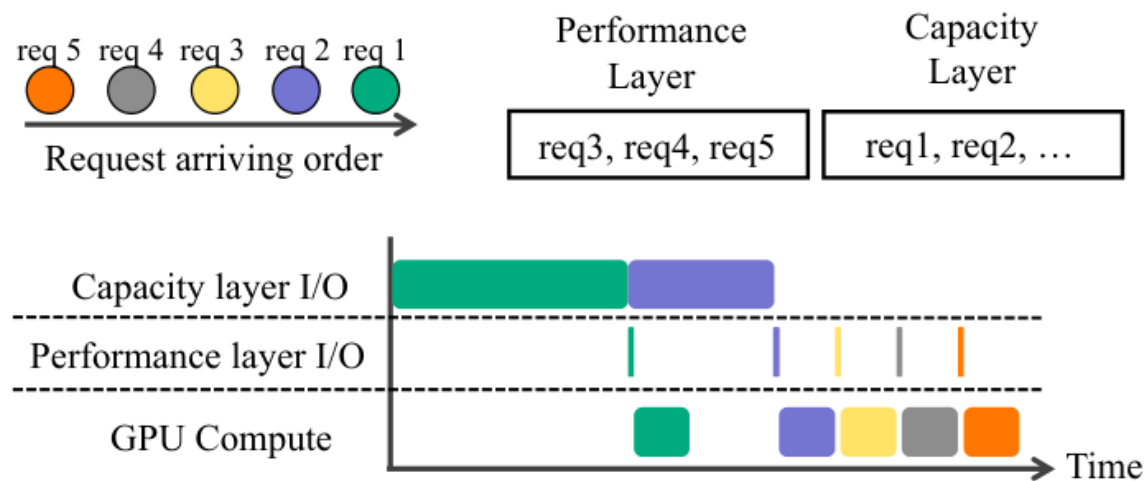
promotion 후에도 **원래 도착 시각을 유지**해 tail latency 급증을 막음

disk-HRRN (preparing queue 정렬)

Response ratio = $1 + (\text{Request waiting time}) / (\text{KV size})$

- ▶ 작은 KV 먼저 → **ready queue** 로 빠르게 promote
- ▶ 대기 시간이 길어지면 큰 KV 도 promote (**starvation 방지**)
- ▶ classical HRRN 을 **I/O time $\approx$ KV size** 에 맞춰 변형

동작 예 — GPU 유휴 시간을 어떻게 없애는가



(a) I/O-oblivious request scheduling (FCFS)



Figure 11. (a) FCFS vs (b) I/O-aware scheduling 의 timeline 비교

(a) FCFS

큐 앞쪽의 req 1, 2(큰 capacity-layer KV) 가 막아서

GPU 와 perf-layer I/O 가 모두 논다

(b) I/O-aware

req 3, 4, 5(perf-layer hit) 가 먼저 GPU 로 가고 , 그 사이 req 2 가 promote

평균 큐잉 시간 **5.76s → 2.45s**
(-57.5%)

Mechanism 2 의 핵심 관찰 — 답변 길이와 다음 reuse distance

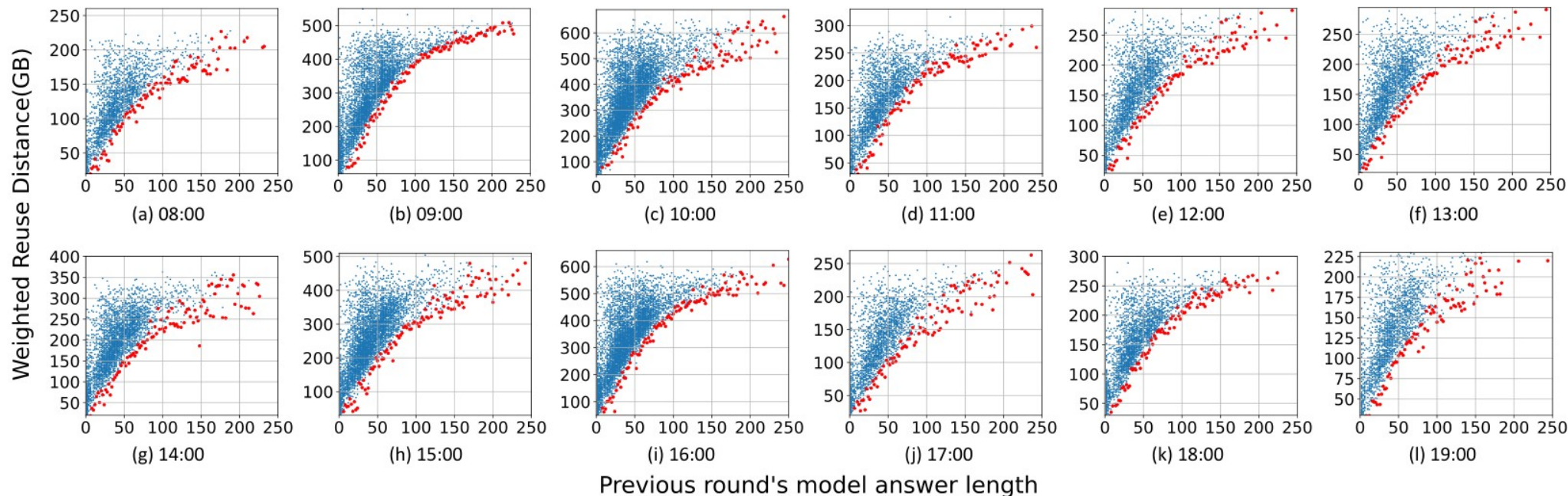


Figure 12. 시간대별 답변 길이(가로축) vs 다음 KV access 의 reuse distance(세로축). 빨간 선 = 하한

정의 — Weighted reuse distance

현재 access 와 다음 access 사이에 접근되는 다른 KV 의 **byte 합**

관찰 — 답변이 길수록 사용자가 다음 질문을 늦게 보내고 , 그 사이 다른 사용자의 KV 가 끼어들면서 reuse distance 하한이 **꾸준히 증가 (Spearman $\rho = 0.94 \sim 0.98$)**

reuse distance 가 크다고 곧 미스인 것은 아니다

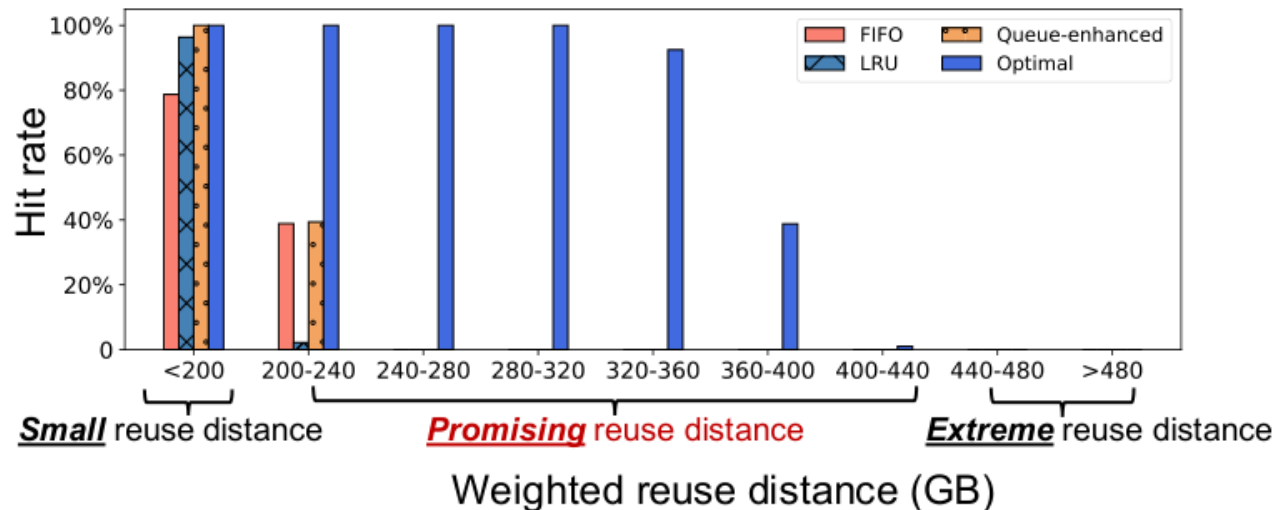


Figure 13. Reuse distance 구간별 hit rate (Optimal vs FIFO/LRU/Q-enh)

세 영역으로 나눔

Small (< perf layer size)

어떤 정책이든 hit ≈ 1.0

Promising (중간 영역)

Optimal(Belady) 은 100% 에 가깝지만
LRU 는 0~40% 수준

Extreme (440 GB+)

Optimal 도 hit ≈ 0 — eviction 1 순위

교훈 — distance 만 보고 evict 하면 promising 구간을 놓친다.
Optimal 과의 격차를 좁히려면 **구간별 hit 확률을 모델링**해야 한다.

Eviction 알고리즘 — Ghost cache + bucket

- ① Promising 영역을 m 개의 fine-grained bucket 으로 나눔
- ② 각 bucket 의 hit rate 를 ghost cache 위에서 Belady 시뮬레이션으로 측정
- ③ 사용자별 과거 KV access 분포로, 다음 access 가 각 bucket 에 떨어질 확률을 추정
- ④ compute 가 보낸 답변 길이로 reuse distance 하한을 정해 더 작은 bucket 의 확률을 0 으로 truncate
- ⑤ Equation 2 로 overall hit potential 을 계산해 가장 낮은 KV 를 evict

Eviction 결정식 — Overall hit potential

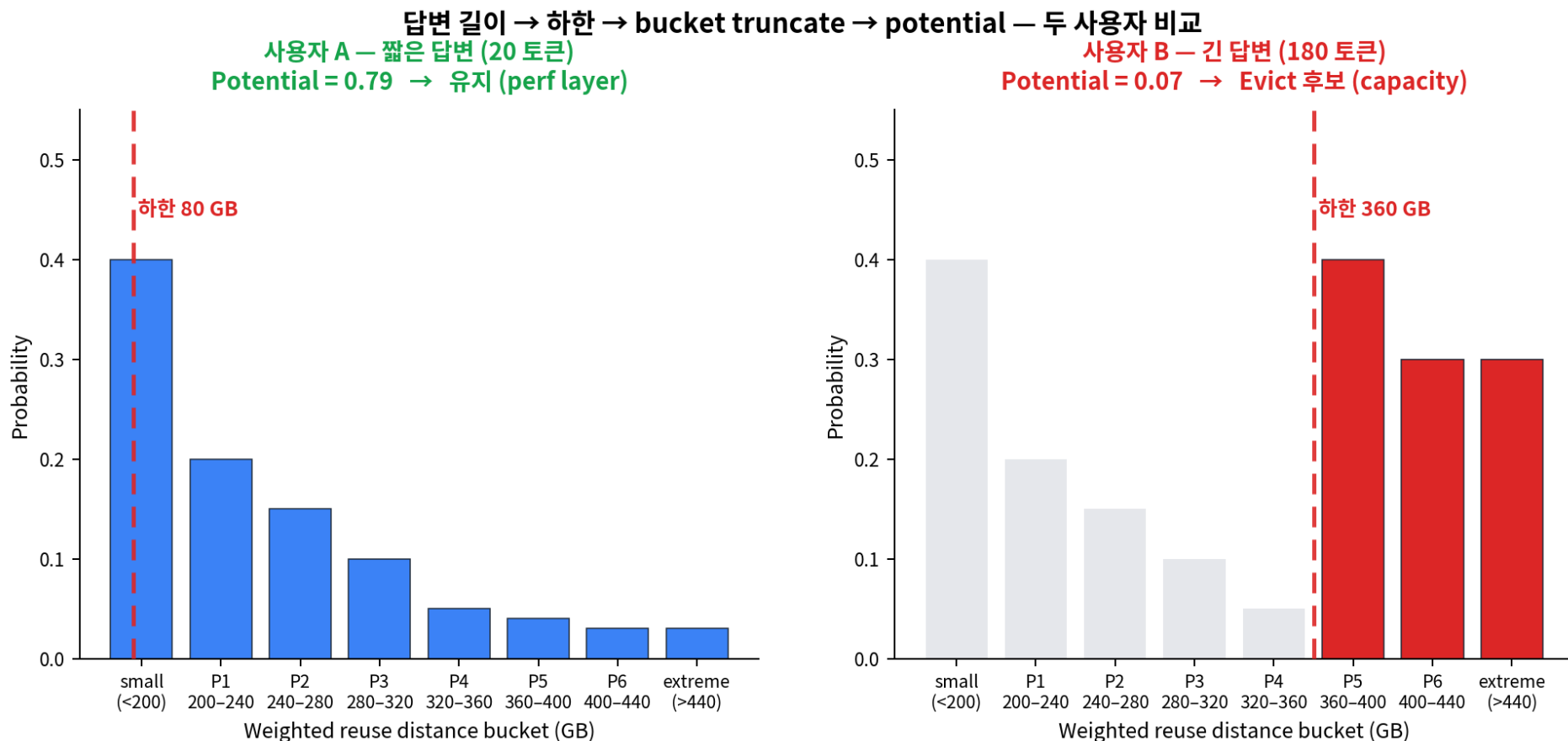
$$\text{Overall_potential} = \text{prob_small} \cdot 1.0 + \text{prob_extreme} \cdot 0.0 + \sum_i \text{prob_promising}(i) \cdot \text{hit_promising}(i)$$

Equation (2). hit potential 이 가장 낮은 KV 를 evict

- ▶ **prob_small** 다음 access 가 perf layer 안에 들어올 확률 — hit 확률 1.0
- ▶ **prob_promising(i)** promising bucket i 에 떨어질 확률 (총 m 개 bucket)
- ▶ **hit_promising(i)** ghost cache 의 Belady 시뮬레이션이 알려주는 bucket i 의 hit rate
- ▶ **prob_extreme** perf layer 로 다시 돌아오지 않음 — hit 확률 0

Note. Ghost cache 는 백그라운드에서 Belady 시뮬레이션을 수행 — eviction 결정 단계의 추가 비용은 **0.35 ms** 에 그침 (자세한 비교는 평가 슬라이드에서)

예시 — 두 사용자의 답변 길이에 따른 evict 결정



같은 과거 분포라도 직전 답변 길이 한 가지가 potential 을 0.79 ↔ 0.07 로 가른다

Mechanism 3 — 어떤 tensor 를 캐싱할 것인가

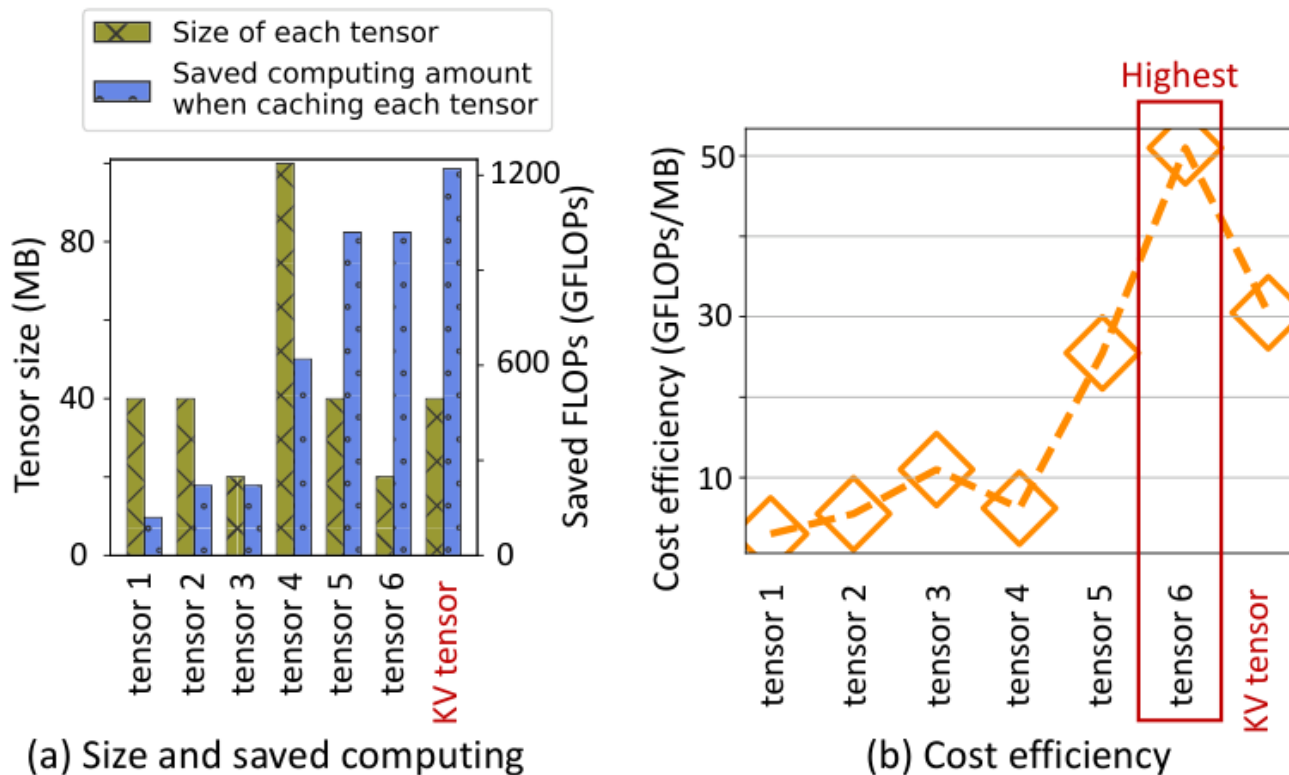


Figure 14. (a) tensor 별 size 와 절약 가능한 GFLOPs, (b) cost efficiency

왜 KV tensor 가 최선이 아닌가

▶ KV tensor 가 직접 필요하지만 **size** 가 크다

Cost Efficiency

= Saved compute / Required space (GFLOPs/MB)

Tensor 6 (정규화된 activation)

▶ 51 GFLOPs/MB — KV tensor(30.5) 의 1.67 배

▶ KV tensor 로 변환하는 데 **GPU 1 step** 이면 충분

Storage-efficient tensor caching — 적용 범위

MHA 기반 모델

- ▶ Llama · Qwen · Bloom · OPT · Baichuan
- ▶ head 마다 K, V 가 분리 → KV 가 큼
- ▶ tensor 6 캐싱이 적은 공간으로 많은 compute 를 절약
- Bidaw 는 **tensor 6** 를 캐싱

GQA 기반 모델

- ▶ 여러 query head 가 K, V 를 공유 → KV tensor 자체가 작음
- ▶ tensor 6 의 cost-efficiency 이점이 사라짐
- GQA 에서는 **KV tensor** 를 그대로 캐싱

구현 노트 — vLLM 위에 있는 네 가지 기법

Continuous batching

일찍 끝난 요청을 배치에서 바로 빼고 새 요청을 곧장 받아들임 (ORCA 계열과 호환)

Mix-grained PagedAttention

history/query 토큰은 256 토큰의 큰 블록, response 토큰은 16 토큰의 작은 블록으로 관리 → CPU-GPU 대역폭 활용도 향상, 단편화 감소

Low-priority CUDA stream

storage-efficient tensor 를 KV tensor 로 바꾸는 작업을 별도 스트림에서 실행 → 유휴 SM 을 활용하고 실제 추론 latency 에 영향 없음

Inclusive caching

eviction 시점에 capacity layer 에 이미 사본이 있어 추가 write 트래픽이 거의 없음 (FlashGen 계열과 동일)

실험 환경 및 비교 대상

Hardware / Software

GPU: NVIDIA A800 80GB

Host mem: 200 GB (perf layer)

Storage: SATA SSD 4 장 RAID-5, 1.5 GB/s

PCIe: Gen 4 (~30 GB/s)

Models: OPT-6.7B/13B/30B, Qwen-7B/14B

Workloads: 자체 100 만 라운드 trace, ShareGPT (Poisson 시뮬)

Baselines

- ▶ **vLLM** Recompute (KV caching 없음)
- ▶ **CachedAttention** Queue-enhanced LRU + 2-tier
- ▶ **FlashGen** Inclusive caching + GPU-fit scheduling
- ▶ **Optimal** 모든 KV 가 perf layer 에 있는 상한선
- ▶ **Bidaw** 이 논문

Overall performance — 모델 5 종 동시 평가

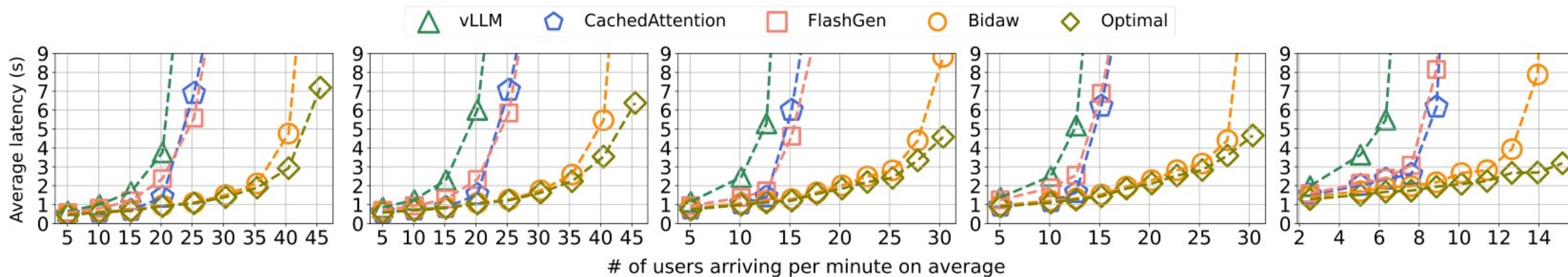


Figure 15. 사용자 도착률 vs 평균 응답 latency, 모델 5 종

- ▶ **Bidaw** — 응답 latency 최대 3.58 배 감소 (vs CachedAttention/FlashGen)
- ▶ **Bidaw** — throughput 1.43~1.83 배 향상 (동일 latency 기준)
- ▶ **Optimal** 과의 격차 거의 없음 — perf-layer-only 수준에 근접
- ▶ **Note** Bidaw 는 lossless — 답변 정확도는 FlashGen 등과 같음

Memory sensitivity & Eviction miss rate

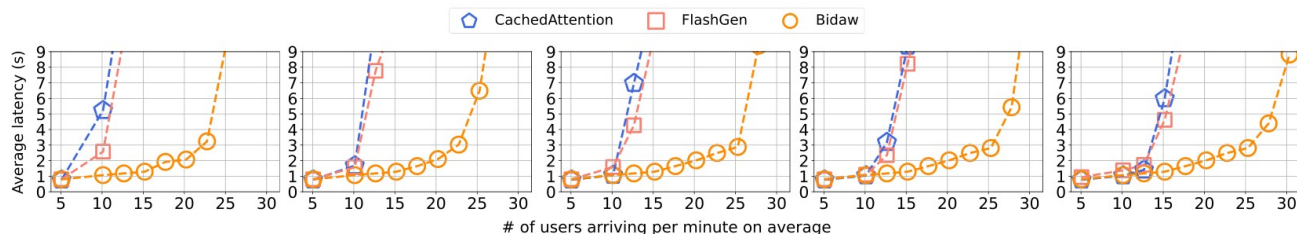


Figure 16. host memory 120~200GB 범위에서의 latency

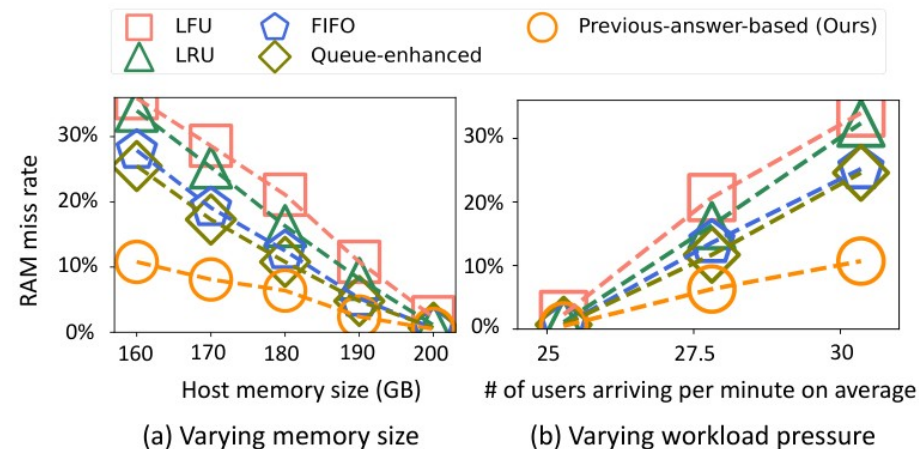


Figure 18. (a) memory size · (b) workload pressure

Memory sensitivity (Fig. 16)

- ▶ host memory 를 120GB 까지 줄여도 Bidaw 는 baseline 200GB 수준의 latency
- ▶ 동일 메모리에서 1.75~2.19 배 더 많은 사용자 / 분 처리 — 메모리가 작을수록 격차가 더 벌어짐

Eviction miss rate (Fig. 18)

- ▶ queue-enhanced (CachedAttention) 보다 miss rate -57.6%
- ▶ LFU/LRU/FIFO 같은 일반 정책 보다 miss rate -69.9%

Tail latency · Ablation · Overhead

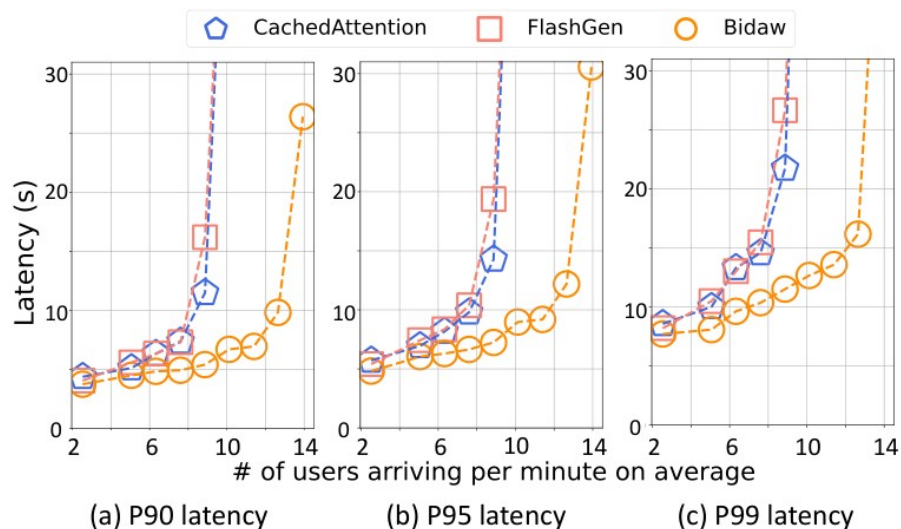


Figure 22. Tail latency P90 / P95 / P99 (OPT-30B)

- ▶ **Tail latency** CachedAttention 대비 P90 -52.96% / P95 -49.30% / P99 -47.03%
- ▶ **Ablation** Scheduling +1.58 배, Eviction +1.25 배, Tensor caching +1.10 배
- ▶ **Overhead** Scheduling 0.62 ms, Eviction 0.35 ms, KV transform <0.1 s

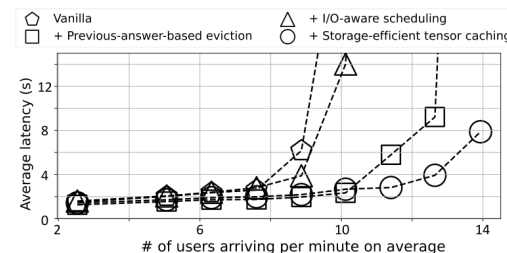


Figure 21. Ablation

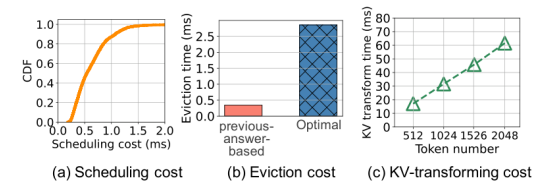


Figure 20. Overhead

정리 · 한계 · 시사점

기여

- ▶ compute 와 storage 사이 양방향 정보 공유라는 설계 원칙을 제시
- ▶ I/O-aware scheduling 과 previous-answer-based eviction 두 메커니즘
- ▶ MHA 에서 storage-efficient tensor 캐싱으로 공간과 연산을 함께 절약
- ▶ 모델 5 종 평균 latency 3.58 배 , throughput 1.83 배 — Optimal 에 근접

한계

- ▶ 한 사용자 대화 내부의 KV 재사용만 다룸 — 사용자 간 공유는 범위 밖
- ▶ GQA 에서는 storage-efficient tensor 의 이점이 사라짐
- ▶ ShareGPT 처럼 timestamp 가 없는 trace 에서는 eviction 효과가 약해짐
- ▶ 단일 GPU 노드 평가 — disaggregated · 멀티노드는 future work

논의

- ▶ 대화 시스템에서 사용자가 글을 읽는 속도가 storage 정책의 유용한 신호가 된다
- ▶ HRRN, Belady ghost cache 같은 고전 기법이 LLM 워크로드에서 다시 유효함을 보임
- ▶ ML serving 과 storage 의 co-design 사례 — 후속 연구가 활발해질 분야